TASKFLOW
SOFTWARE SPECIFICATION

Specification-Driven Development (SDD) Report

Task creation, organization, prioritization, and progress tracking

This document is the proposed source of truth for the TaskFlow implementation.

Purpose and Intended Outcome

TaskFlow is a task management web application that allows users to create, view, update, delete, and organize tasks. Users can assign a priority and status to each task, optionally set a due date, and filter tasks to understand what needs attention. This specification defines the initial release behavior, implementation boundaries, API interfaces, and verification criteria for AI-assisted engineering.

This is a proposed specification based on the TaskFlow project idea discussed for practicing Specification-Driven Development. It is not extracted from the example IT Helpdesk PDF. Items marked P are proposed baseline decisions and can be revised before implementation.

Document Map

1. Scope and Baseline Assumptions

1.1 Included in the initial release

• Create a task with a title and optional description, priority, status, and due date.

• View all tasks and retrieve an individual task by ID.

• Update task details and delete a task.

• Filter tasks by status and priority and search by title.

• Persist tasks in a database so they remain available after application restart.

• Expose a web interface and REST API with validation and useful error states.

1.2 Requirements baseline

1.3 Proposed assumptions

• The first release is a single-user or local demonstration application; user accounts and multi-user permissions are not included unless later requested.

• The proposed frontend is React, backend is FastAPI, and persistence uses PostgreSQL for persistent storage.

• Priority values are Low, Medium, and High. The default priority is Medium.

• Status values are Pending, In Progress, and Completed. The default status is Pending.

• The due date is optional. If provided, it must be a valid date.

• Task deletion is allowed in this project baseline; confirmation should be shown in the UI.

1.4 Excluded from this release

User registration/login, role-based access control, team assignment, recurring tasks, subtasks, file attachments, email or push notifications, calendar synchronization, analytics dashboards, and AI-generated task suggestions are excluded unless added through an approved specification change.

2. Actors and Access

The initial baseline assumes one local user interacting with TaskFlow. Because authentication and multi-user access are out of scope, the API does not claim to provide user-level authorization. If the project is extended for multiple users, identity, ownership, permissions, and access-control tests must be specified before implementation.

3. Functional Requirements

4. Task Workflow and Business Rules

No stricter task lifecycle is imposed in version 1.0. For example, the baseline does not prohibit editing a Completed task or moving a task from Completed back to In Progress. If such restrictions are desired, they must be added as explicit requirements with corresponding acceptance criteria and tests.

5. Web Interface Specification

5.1 Form and usability rules

• Client-side validation should provide quick feedback, but the backend remains authoritative.

• On validation or network failure, preserve user-entered form values.

• Do not show success until the server confirms the operation.

• Use readable labels, visible keyboard focus, and status text rather than relying only on color.

• After create, update, or delete, refresh the list so the visible UI reflects persisted data.

6. Data Model and Validation

The proposed title and description limits are implementation decisions for this draft, not user-confirmed requirements. They should be retained or revised before implementation. The database should enforce uniqueness for IDs and valid enumerated values where supported.

7. REST API Contract

Base path: /api. Requests and responses use application/json.

7.1 Example request and response

POST /api/tasks — request:

{
"title": "Prepare project report",
"description": "Complete the SDD report and review acceptance criteria",
"priority": "High",
"status": "Pending",
"due_date": "2026-11-05"
}

Example successful response (server-generated values are illustrative):

{
"id": 1,
"title": "Prepare project report",
"description": "Complete the SDD report and review acceptance criteria",
"priority": "High",
"status": "Pending",
"due_date": "2026-11-05",
"created_at": "2026-10-29T06:30:00Z",
"updated_at": "2026-10-29T06:30:00Z"
}

7.2 Error handling

The implementation should use a consistent error response format. Exact response schemas and whether the framework uses 400 or 422 for particular parsing/validation failures should be finalized in the API implementation contract.

8. Quality and Operational Requirements

8.1 Architecture boundaries

The React frontend calls the FastAPI REST API. The API owns validation and task operations. A relational database (PostgreSQL) stores tasks. The frontend must not connect directly to the database. This is a proposed stack for the exercise; the core behavior and API contract should remain the source of truth.

9. Acceptance Criteria and Test Cases

Each acceptance criterion should be verified by one or more tests. The tests below form the initial baseline and should be automated where practical.

9.1 Release acceptance

• All accepted requirements must map to one or more passing tests.

• Create, list, retrieve, update, delete, filtering, and persistence flows must work end to end.

• Invalid input must not create or corrupt records.

• The UI must show the persisted server state and handle errors without false success.

• Record test execution results; this specification does not claim that an implementation has already passed.

10. Implementation Plan

11. SDD Workflow and Change Control

11.1 AI-assisted implementation workflow

– Ask the AI to review this specification and identify ambiguity before coding.

– Break the work into small tasks, each linked to requirement IDs and test cases.

– Prompt the AI to implement only the approved task and not add features outside scope.

– Generate or update tests from the acceptance criteria and execute them.

– Review generated code, test results, error handling, and whether the implementation matches the specification.

– If a requirement changes, update this document and affected tests before modifying code.

11.2 Change control

Store this specification alongside the code in version control, for example as SPEC.md or a project document. For each approved change, record the date, rationale, affected requirement IDs, interface/data changes, and updated tests. A later AI prompt does not override the specification unless the requirement change is explicitly approved and recorded.

12. Open Decisions

13. Conclusion

TaskFlow is a practical project for applying Specification-Driven Development because it includes clear CRUD behavior, data validation, REST interfaces, persistence, filtering, and testable acceptance criteria. The specification serves as the reference for AI-generated implementation work, while prompts are used to request individual tasks. The implementation is considered complete only when its behavior is verified against the accepted requirements and test cases.

Definition of Ready

Before implementation begins, confirm the technology stack, ID format, field-length limits, due-date policy, and error response format. Once those decisions are approved, implement the API and tests in small increments, then connect the React frontend.

Project	TaskFlow — Task Management System
Version	1.0
Date	29 October 2026

Project	TaskFlow — Task Management System
Status	Draft specification for implementation and testing

Section	Coverage
1–2	Scope, assumptions, actors, and access
3–4	Functional requirements and task workflow
5–6	User interface, data model, and validation
7–8	REST API, errors, and reliability
9–10	Quality requirements, acceptance criteria, and tests
11	Implementation plan, change control, and open decisions

ID	Basis	Requirement
TF-S01	P	Users shall be able to create a task with a required title.
TF-S02	P	Users shall be able to list tasks and retrieve a task by ID.
TF-S03	P	Users shall be able to update task details.
TF-S04	P	Users shall be able to delete a task.
TF-S05	P	Tasks shall have a priority of Low, Medium, or High.
TF-S06	P	Tasks shall have a status of Pending, In Progress, or Completed.
TF-S07	P	Users shall be able to filter tasks by status and priority and search by title.
TF-S08	P	Invalid data shall be rejected with a clear validation response.
TF-S09	P	Tasks shall persist across application restarts.
TF-S10	P	Verification shall include valid, invalid, not-found, and persistence scenarios.

Capability	Initial release behavior
Create task	Available through the web form and POST endpoint.
List tasks	Return saved tasks, with optional filters and search.
Read task	Return one task by ID or a not-found response.
Update task	Update supported fields after validation.
Delete task	Delete an existing task after UI confirmation.
Filter/search	Filter by status/priority and search title.

ID	Requirement
FR01	The system shall create a task with a unique server-generated ID, title, optional description, priority, status, optional due date, and timestamps.
FR02	Title is required and shall not be empty or whitespace-only.
FR03	The system shall display a list of saved tasks, including title, status, priority, and due date when present.
FR04	The system shall retrieve a task by ID and return a not-found response for an unknown ID.
FR05	The system shall allow supported task fields to be updated after validation.
FR06	The system shall delete a task only when the specified ID exists; the UI shall ask for confirmation.
FR07	The system shall accept only the defined priority and status values.
FR08	The system shall filter tasks by status and priority and search titles case-insensitively.
FR09	The system shall persist task data in the configured database.

ID	Requirement
FR10	The UI shall show loading, empty, success, and error states where applicable.

Operation	Rule
Create	The server generates the ID and timestamps. If priority or status is omitted, apply the documented defaults: Medium and Pending.
Update	Only supported fields may be updated. Unknown fields shall be rejected. updated_at changes after a successful update.
Priority	Allowed values: Low, Medium, High.
Status	Allowed values: Pending, In Progress, Completed. This baseline permits direct updates among these values.
Due date	Optional. If supplied, it must be a valid date in the agreed format.
Delete	The UI asks for confirmation. A successful deletion removes the task; deleting an unknown ID returns not found.
Filtering	When multiple filters are provided, they combine using AND. Search matches title case-insensitively.
Persistence	A successful create or update must survive an application restart.

Screen/component	Required content and behavior
Task Dashboard	List tasks with title, status, priority, and optional due date. Provide create action, search, and filters.
Create Task form	Title, description, priority, status, optional due date, and submit action. Title is visibly required.
Task Details/Edit	Show task fields and allow editing supported values.
Delete action	Ask for confirmation before deleting. Cancel leaves the task unchanged.
Empty state	Display a message such as “No tasks found” and allow filters/search to be cleared.
Loading/error state	Show loading feedback during requests and a useful error with retry where appropriate.

Field	Type and rule
id	Integer or UUID generated by the server; unique and immutable.
title	Required string; proposed length 1–150 characters after trimming.
description	Optional string; proposed maximum length 2,000 characters.
priority	Enum: Low, Medium, High; default Medium.
status	Enum: Pending, In Progress, Completed; default Pending.
due_date	Optional valid date; use a consistent ISO 8601 date format.
created_at	Server-generated timestamp; immutable.
updated_at	Server-generated timestamp; changes after successful update.

Method / endpoint	Purpose and expected behavior
GET /tasks	List tasks. Optional query parameters: status, priority, q. Return 200 and a list of matching tasks.
GET /tasks/{id}	Return one task by ID. Return 404 if it does not exist.
POST /tasks	Create a task. Validate fields and return 201 with the saved task.
PUT /tasks/{id}	Update supported task fields. Return 200 with the updated task, or 404 if missing.
DELETE /tasks/{id}	Delete the task. Return 204 on success, or 404 if missing.

Status	Meaning
400	Malformed request body or invalid request format.
404	Task ID does not exist.
422	Field validation failed or an unsupported enum value was supplied.

Status	Meaning
500	Unexpected server error; response must not expose stack traces or database credentials.

ID	Requirement and verification
NFR01 Reliability	Successful task writes persist in the configured database and survive application restart.
NFR02 Validation	The backend validates all input regardless of frontend checks.
NFR03 Security	Do not expose secrets, database credentials, or stack traces in responses or source control.
NFR04 Usability	Primary actions remain usable on desktop and mobile widths; forms have labels and visible errors.
NFR05 Maintainability	Separate frontend components, API routes, validation/schema logic, and database access responsibilities.
NFR06 Testability	Automate tests for CRUD operations, validation, filters, not-found responses, and persistence.
NFR07 Consistency	The UI must reflect the server's saved state and must not display false success after failed requests.

Test ID	Given/action	Expected result and trace
TC01	Submit a task with a valid title and supported field values.	201; task is saved and returned. FR01, FR02.
TC02	Submit a missing, empty, or whitespace-only title.	Validation error; no task is created. FR02.
TC03	Submit an unsupported priority such as Urgent.	Validation error; no invalid value is stored. FR07.
TC04	Submit an unsupported status.	Validation error; no invalid value is stored. FR07.
TC05	Request the task list.	200; saved tasks are returned. FR03.
TC06	Retrieve an existing task by ID.	200; correct task returned. FR04.
TC07	Retrieve a nonexistent task ID.	404; no unrelated data is returned. FR04.
TC08	Update an existing task with valid values.	200; updated values are persisted. FR05.
TC09	Update a nonexistent task.	404; no new task is created. FR05.
TC10	Delete an existing task.	204; task no longer appears in list. FR06.
TC11	Delete a nonexistent task.	404; response is consistent. FR06.
TC12	Filter tasks by status and priority.	Only tasks matching all supplied filters are returned. FR08.
TC13	Search using part of a title with different letter casing.	Matching tasks are returned case-insensitively. FR08.
TC14	Restart the application after creating a task.	Task remains available. FR09.
TC15	Cause a backend/API failure during a form submission.	Error is displayed; entered values are retained and success is not shown. FR10.
TC16	Cancel deletion in the confirmation dialog.	Task remains unchanged. UI requirement.

Phase	Work	Verification focus
1. Specification review	Confirm field rules, status/priority defaults, error format, and selected stack.	Resolve ambiguity before coding.
2. Backend and data	Create task schema, database persistence, and CRUD endpoints.	TC01–TC11; validation and persistence.
3. Filtering/search	Add status/priority filters and title search.	TC12–TC13; combinations and empty results.
4. React UI	Build dashboard, create/edit forms, task details, delete confirmation, and error states.	TC15–TC16; UI matches API behavior.
5. Integration and regression	Connect UI to API and run the complete test suite.	All acceptance criteria pass; no regression.

Decision	Proposed baseline	Impact if changed
Frontend/backend	React + FastAPI	Changes project structure and implementation approach.
Database	PostgreSQL	Changes setup, migrations, and database integration.
ID format	Integer or UUID; select one before implementation.	Affects route schemas, UI links, and tests.
Title/description limits	Title 1–150; description up to 2,000 characters.	Must be consistent across UI, API, and database.
Due date policy	Optional ISO-format date.	Timezone and past-date policy should be clarified if required.
Authentication	Excluded from version 1.0.	Multi-user use requires identity, ownership, and authorization rules.
Status transitions	Direct changes among the three statuses are allowed.	Restricting transitions requires new business rules and tests.